

Reviewer Pack — Minimal Rank of Geometric Algebraic Curves and Communication...

Entry 2e3287498df8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 09:00:59 UTC

Minimal Rank of Geometric Algebraic Curves and Communication Complexity Rank Variance

Entry ID: 2e3287498df8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 09:00:59 UTC

1. Conjecture

Field A (mathematical branch): Geometric Algebra **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

The minimal rank of the algebraic curves defined over the field of two elements corresponding to a given Boolean function is linearly correlated with the communication complexity rank variance of its associated protocol, such that $\text{RANK}(\text{curve}) = \Theta(\text{RankVar}(\text{protocol}))$.

Rationale (proposer's reasoning):

Geometric algebra provides an alternative way to represent boolean functions through algebraic structures. These representations might expose underlying geometric properties that are not apparent in classical circuit representations, potentially leading to new insights into the complexity of communication protocols.

Taxonomy category: geometric_algebra (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 85781a1dd7104fd5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between the minimal rank of the geometric algebraic curves and the communication complexity rank variance exceeds 0.8 for at least 80% of the seeds, with an absolute difference in ranks of no more than 3 across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Geometric Algebra AND Communication Complexity Rank Variance - Boolean Function Geometric Algebraic Curves Communication Complexity - Algebraic Curve Rank Geometric Algebra Correlation Communication Complexity Matrix Rank Variance

Top relevant hits considered: - [<http://arxiv.org/abs/2105.12016v4>] Waring ranks of sextic binary forms via geometric invariant theory - [<http://arxiv.org/abs/2510.02583v2>] The Log-Rank Conjecture: New Equivalent Formulations - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/2407.04826v1>] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits - [<http://arxiv.org/abs/1407.6169v3>] Multiplicative Complexity of Vector Valued Boolean Functions - [<http://arxiv.org/abs/1403.8106v1>] Recent advances on the log-rank conjecture in communication complexity - [<http://arxiv.org/abs/2002.00568v4>] Tautological algebra of the moduli stack of semistable bundles of rank 2 on a general curve

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def construct_algebraic_curve(f):
        n = len(f)
        curve = []
        for x in range(2**n):
            y = f[x]
            if y == 0:
                curve.append(x)
            else:
                curve.append(-x)
        return curve

    def communication_complexity_rank_variance(curve):
        n = len(curve)
```

```

matrix = [[0] * n for _ in range(n)]
for i in range(n):
    for j in range(i, n):
        if abs(curve[i]) == abs(curve[j]):
            matrix[i][j] = 1
            matrix[j][i] = 1
rank = gaussian_elimination(matrix)
return rank

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        if A[i][i] == 0:
            continue
        for j in range(n):
            A[i][j] /= A[i][i]
        for j in range(m):
            if j != i and A[j][i] != 0:
                factor = A[j][i]
                for k in range(n):
                    A[j][k] -= factor * A[i][k]
    rank = sum(1 for row in A if any(row))
    return rank

def pearson_correlation(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    cov = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n)) / n
    std_x = math.sqrt(sum((x[i] - mean_x)**2 for i in range(n)) / n)
    std_y = math.sqrt(sum((y[i] - mean_y)**2 for i in range(n)) / n)
    return cov / (std_x * std_y)

def correlation_check(x, y):
    if len(x) != len(y):
        return False
    corr = pearson_correlation(x, y)
    return abs(corr) > 0.8

n_values = [5, 10, 15, 20, 30, 40]
curve_ranks = []
rank_variances = []

for n in n_values:
    f = generate_boolean_function(n)
    curve = construct_algebraic_curve(f)
    rank_variance = communication_complexity_rank_variance(curve)
    curve_ranks.append(len(curve))
    rank_variances.append(rank_variance)

if not correlation_check(curve_ranks, rank_variances):

```

```

    return {
        "metric_name": "Pearson Correlation",
        "metric_value": pearson_correlation(curve_ranks, rank_variances),
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "Correlation check failed"
    }

    return {
        "metric_name": "Pearson Correlation",
        "metric_value": pearson_correlation(curve_ranks, rank_variances),
        "instances_tested": len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }
}

if __name__ == "__main__":
    if not sys.argv[1:]:
        seeds = [2*i + 3 for i in range(5, 8)] # Default list of 30 primes
    else:
        seeds = [int(seed) for seed in sys.argv[1:]]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={seeds[seeds.index(counterexample)]}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fdc0e703.py", line 119, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fdc0e703.py", line 87, in run_trial

```

    curve = construct_algebraic_curve(f)

```

